

Week 1 | Python Programming Lab Tasks

CLO – 1: Explain how AI enables intelligent behavior in real-world applications such as game playing, autonomous vehicles, and robotic systems
(Students will apply Basic Python programming concepts to implement computational components of real-world intelligent systems)

Task 1: Smart Parking Space Analysis

Objective

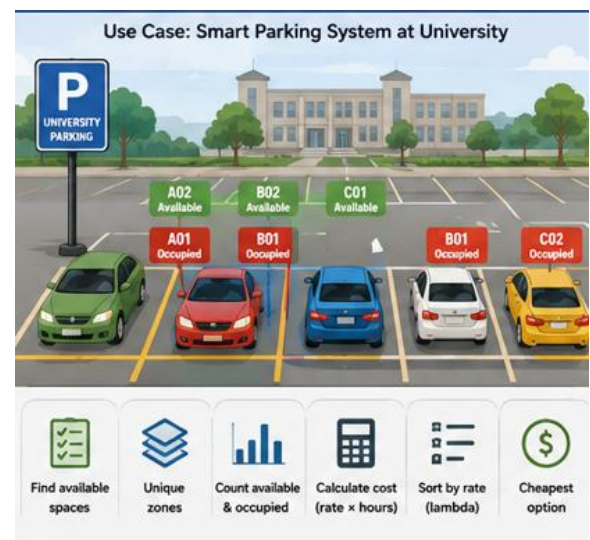
To apply Python lists, dictionaries, sets, conditions, loops, functions, and lambda functions to analyze parking-space information.

Scenario

A smart parking application monitors parking spaces in a university parking facility. Each parking space has a unique ID, zone, current status, and hourly parking rate.

Use the following data:

```
parking_data = [  
    {"id": "A01", "zone": "A", "status": "occupied", "rate": 100},  
    {"id": "A02", "zone": "A", "status": "available", "rate": 100},  
    {"id": "B01", "zone": "B", "status": "occupied", "rate": 80},  
    {"id": "B02", "zone": "B", "status": "available", "rate": 80},  
    {"id": "C01", "zone": "C", "status": "available", "rate": 120},  
    {"id": "C02", "zone": "C", "status": "occupied", "rate": 120}  
]
```



Requirements

Write a Python program that:

1. Displays all currently available parking spaces.
2. Uses a **set** to store the unique parking zones.
3. Counts the number of available and occupied spaces.
4. Creates a function:

```
def calculate_cost(rate, hours):
```

...

to calculate the parking cost for a given number of hours.

5. Uses a **lambda function with sorted()** to rank available spaces according to their hourly rate.
6. Allows the user to enter the number of parking hours and displays the estimated cost for each available space.
7. Displays the cheapest available parking option.

This represents the data-processing component of an intelligent parking system that can later be extended to recommend parking spaces based on availability, cost, and location.

Concepts Practiced

Lists, dictionaries, sets, conditions, loops, functions, lambda functions, sorting, user input

Task 2: Cybersecurity Login Risk Analyzer

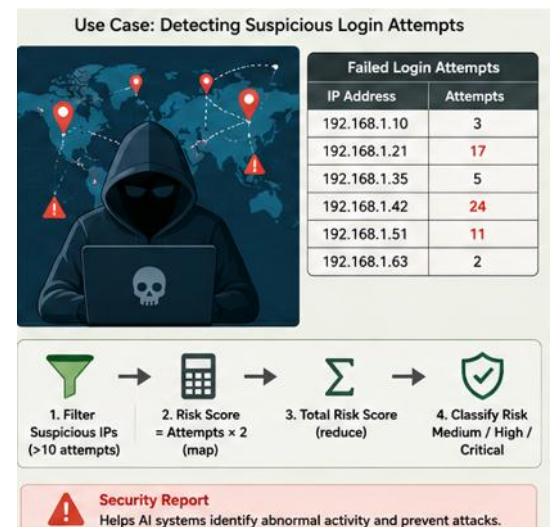
Objective

To apply functional programming, dictionaries, conditions, and aggregation to identify suspicious login behavior.

Scenario

A cybersecurity system records failed login attempts from different IP addresses during a monitoring period.

```
login_data = {  
    "192.168.1.10": 3,  
    "192.168.1.21": 17,  
    "192.168.1.35": 5,  
    "192.168.1.42": 24,  
    "192.168.1.51": 11,  
    "192.168.1.63": 2  
}
```



An IP address generating more than **10 failed attempts** is considered suspicious.

Requirements

Write a Python program that:

1. Uses filter() with a **lambda function** to identify suspicious IP addresses.
2. Calculates a risk score for every suspicious IP using:

Risk Score = Failed Attempts × 2

3. Uses map() with a lambda function to generate the risk scores.
4. Uses reduce() to calculate the total risk score generated by all suspicious IPs.
5. Stores the suspicious IP addresses in a **set**.
6. Displays the IP address with the highest number of failed attempts.
7. Classifies each suspicious IP as:

11–15 → Medium Risk

16–20 → High Risk

Above 20 → Critical Risk

8. Displays a final security report.

This is a simplified version of the preprocessing performed in intrusion-detection and anomaly-detection systems, where raw logs are converted into meaningful features before an AI model identifies abnormal behavior.

Concepts Practiced

Dictionaries, sets, conditions, functions, lambda, filter, map, reduce, loops

Task 3: Personalized Book Recommendation Analyzer

Objective

To use Python data structures and NumPy to calculate and rank recommendation scores.

Scenario

A digital library wants to recommend books to users based on three factors:

- user rating,
- number of times the book was viewed,
- popularity score.

Represent the data using NumPy:

```
import numpy as np
```

```
books = np.array([  
    [4.5, 120, 80],  
    [4.2, 200, 65],  
    [4.8, 150, 95],  
    [3.9, 300, 55],  
    [4.6, 180, 88]  
])
```

Each row represents a book:

Column 0 → Rating

Column 1 → Views

Column 2 → Popularity

Requirements

Write a Python program that:

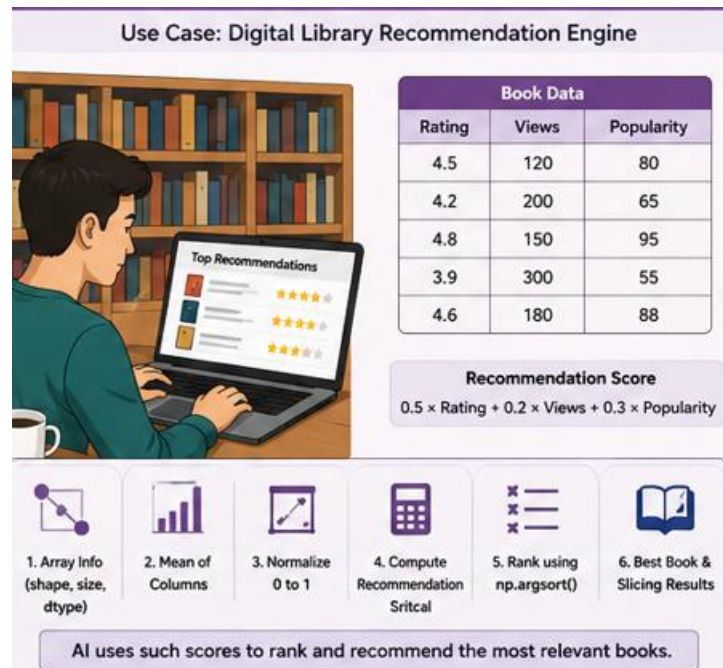
1. Displays the shape, size, and data type of the NumPy array.
2. Calculates the mean value of each column.
3. Normalizes each column between 0 and 1 using Min–Max normalization.
4. Calculate a recommendation score:

Recommendation Score = $0.5 \times \text{Rating} + 0.2 \times \text{Views} + 0.3 \times \text{Popularity}$

5. Uses `np.argsort()` to rank the books from highest to lowest recommendation score.
6. Displays the ranking and corresponding recommendation scores.
7. Identifies the book with the highest recommendation score.
8. Uses array slicing to display the rating and popularity information separately.

Recommendation systems convert different user/item characteristics into numerical features and combine them to rank candidate items.

Concepts Practiced



NumPy arrays, slicing, shape, size, data type, mean, normalization, vectorized operations, argsort(), indexing

NOTE:

Normalization means **rescaling data to a common range**, usually **0 to 1**, so that:

- Large values do not dominate smaller values
- Different features become comparable
- Algorithms behave more stably and fairly

Simple idea:

Convert values like [10, 50, 100] into something like [0.0, 0.44, 1.0]

Why do we normalize data?

Imagine two features:

Feature	Range
Age	0 – 100
Income	10,000 – 1,000,000

If we don't normalize:

- Income will dominate calculations
- Distance-based methods behave incorrectly
- Learning becomes unstable

Normalization ensures all features contribute equally.

Most Common Normalization Method (Min–Max Normalization)

Formula:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

What this does:

- Smallest value $\rightarrow 0$
- Largest value $\rightarrow 1$
- Everything else $\rightarrow$ between 0 and 1

Task 4: Smart Agriculture Field Monitoring

Objective

To combine lists, dictionaries, NumPy, statistical analysis, and decision logic to identify fields requiring attention.

Scenario

An agricultural monitoring system receives measurements from different crop fields.

```
field_data = [
```

```
{
    "field": "F1",
    "temperature": 34,
    "humidity": 55,
    "soil_moisture": 28
```

```
},
```

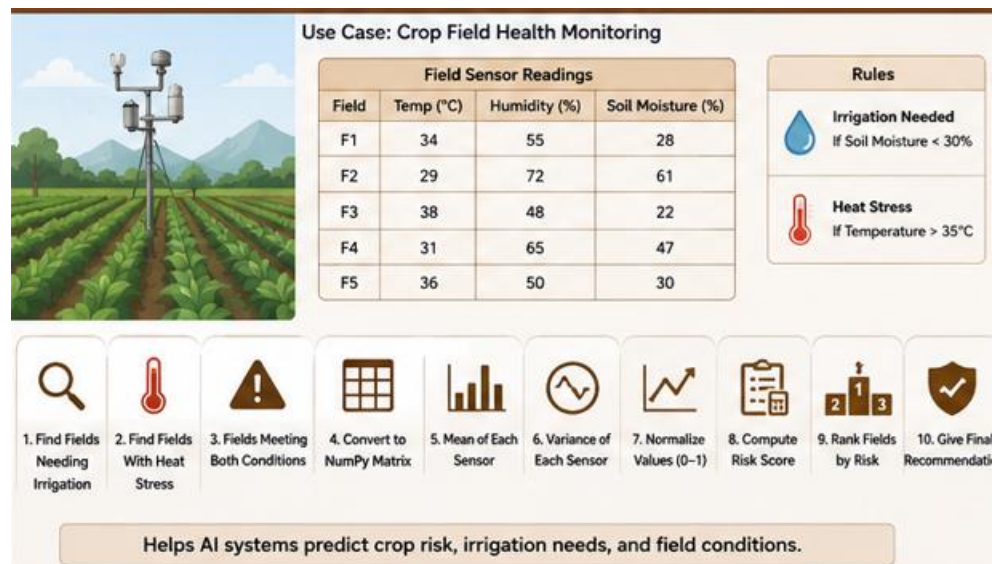
```
{
    "field": "F2",
    "temperature": 29,
    "humidity": 72,
    "soil_moisture": 61
```

```
},
```

```
{
    "field": "F3",
    "temperature": 38,
    "humidity": 48,
    "soil_moisture": 22
```

```
},
```

```
{
    "field": "F4",
```



```
"temperature": 31,  
"humidity": 65,  
"soil_moisture": 47  
},  
{  
"field": "F5",  
"temperature": 36,  
"humidity": 50,  
"soil_moisture": 30  
}  
]
```

A field requires irrigation attention when:

soil moisture < 30

and is considered under **heat stress** when:

temperature > 35

Requirements

Write a Python program that:

1. Identifies fields requiring irrigation.
2. Identifies fields experiencing heat stress.
3. Identifies fields satisfying **both conditions**.
4. Converts the sensor values into a NumPy matrix where:
 - rows represent fields;
 - columns represent temperature, humidity, and soil moisture.
5. Calculates:
 - mean temperature,
 - mean humidity,

- mean soil moisture.
- 6. Calculates the variance of each sensor.
- 7. Identifies the sensor with the highest variance.
- 8. Calculates a simple **field-risk score**:

Risk Score = $0.4 \times \text{normalized temperature} + 0.2 \times \text{normalized humidity} + 0.4 \times (1 - \text{normalized soil moisture})$

- 9. Ranks fields from highest to lowest risk.
- 10. Displays a recommendation:
 - High Risk → Immediate inspection required
 - Medium Risk → Monitor field
 - Low Risk → Normal condition

This represents the preprocessing and rule-based decision layer of an intelligent agriculture system. The same sensor features could later be provided to a machine-learning model for crop-risk prediction.

Concepts Practiced

Dictionaries, lists, conditions, loops, functions, NumPy, normalization, mean, variance, indexing, ranking

Task 5: Intelligent Public Transport Route Analyzer

Objective

To integrate Python data structures, functions, sets, tuples, recursion, and NumPy into a small AI-oriented route-analysis system.

Scenario

A public transportation application stores information about several bus stops and possible routes.

Each stop is represented using a tuple containing its name and approximate coordinates.

```
stops = [  
    ("Stop A", (33.6425, 73.1730)),  
    ("Stop B", (33.6500, 73.1600)),  
    ("Stop C", (33.6550, 73.1500)),
```

```
("Stop D", (33.6600, 73.1400)),  
("Stop E", (33.6650, 73.1300))  
]
```

Possible routes are represented as nested lists:

```
routes = [  
    ["Stop A", "Stop B", ["Stop C", "Stop D"]],  
    ["Stop A", ["Stop E"]]  
]
```

Requirements

Write a Python program that:

Part A: Route Representation

1. Use **tuples** to represent the coordinates of each stop.
2. Use a **dictionary** to map every stop to its coordinates.

Example:

```
{  
    "Stop A": (33.6425, 73.1730)  
}
```

Part B: Route Exploration

3. Use a **set** to maintain the stops that have already been visited.
4. Write a function that checks whether a stop has already been visited.
5. Add newly visited stops to the set.

Part C: Nested Route Processing

6. Write a **recursive function** to flatten the nested routes structure into a single list of stops.

Part D: Distance Calculation

7. Write a function:

```
def calculate_distance(point1, point2):
```

...

to calculate the Euclidean distance between two coordinate points.

Part E: NumPy Analysis

8. Store the coordinates of all stops in a NumPy array.
9. Calculate:
 - mean latitude,
 - mean longitude,
 - minimum latitude,
 - maximum latitude.
10. Calculate the distance between consecutive stops.

Part F: Route Decision

11. Identify the closest pair of consecutive stops.
12. Display the complete flattened route.
13. Display the visited stops.
14. Display the calculated route statistics.

*This task models several components of an intelligent transportation system. The same representations can later be extended into graph-based search algorithms such as **Breadth-First Search**, **Depth-First Search**, **Uniform Cost Search**, and **A***.*

Concepts Practiced

Lists, nested lists, tuples, dictionaries, sets, recursion, functions, NumPy, arrays, indexing, loops, mathematical operations

